

Exercise 1: Mocking Databases and Repositories

Code:

Repository.java

```
package com.Mockito;

public interface Repository {

    String getData();

}
```

Service.java

```
package com.Mockito;

public class Service {

    private Repository repository;

    public Service(Repository repository) {
        this.repository = repository;
    }

    public String processData() {
        return "Processed " + repository.getData();
    }

}
```

ServiceTest.java

```
package com.Mockito;

import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.Test;

public class ServiceTest {

    @Test
    public void testServiceWithMockRepository() {

        // Create a mock repository
        Repository mockRepository = mock(Repository.class);

        // Stub the repository method
        when(mockRepository.getData()).thenReturn("Mock Data");

        // Create the service object
```

```

Service service = new Service(mockRepository);

// Call the service method
String result = service.processData();

// Verify the result
assertEquals("Processed Mock Data", result);
}
}

```

Output:

The screenshot shows an IDE with a project named 'Mockito'. The left sidebar shows the project structure with a 'test' directory containing a 'ServiceTest' class. The main editor shows the code for 'ServiceTest.java', which includes imports for Mockito, JUnit, and Assertions, and a test method 'testServiceWithMockRepository()' that creates a mock repository, stubs its 'getData()' method to return 'Mock Data', and then creates a 'Service' object. The bottom panel shows the test results: 'ServiceTest (com.Mocki) 1 sec 134 ms' with '1 test passed' and '1 test total, 1 sec 134 ms'. The console output shows the command 'C:\Program Files\Eclipse Adoptium\jdk-17.0.18-hotspot\bin\java.exe' and a warning from OpenJDK 64-Bit Server VM, followed by 'Process finished with exit code 0'.

Exercise 2: Mocking External Services (RESTful APIs)

Code:

RestClient.java

```

package com.Mockito;

public interface RestClient {

    String getResponse();

}

```

ApiService.java

```
package com.Mockito;

public class ApiService {

    private RestClient restClient;

    public ApiService(RestClient restClient) {
        this.restClient = restClient;
    }

    public String fetchData() {
        return "Fetched " + restClient.getResponse();
    }

}
```

ApiServiceTest.java

```
package com.Mockito;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.mock;
import static org.mockito.Mockito.when;

import org.junit.jupiter.api.Test;

public class ApiServiceTest {

    @Test
    public void testServiceWithMockRestClient() {

        // Step 1: Create a mock REST client
        RestClient mockRestClient = mock(RestClient.class);

        // Step 2: Stub the method
        when(mockRestClient.getResponse())
            .thenReturn("Mock Response");

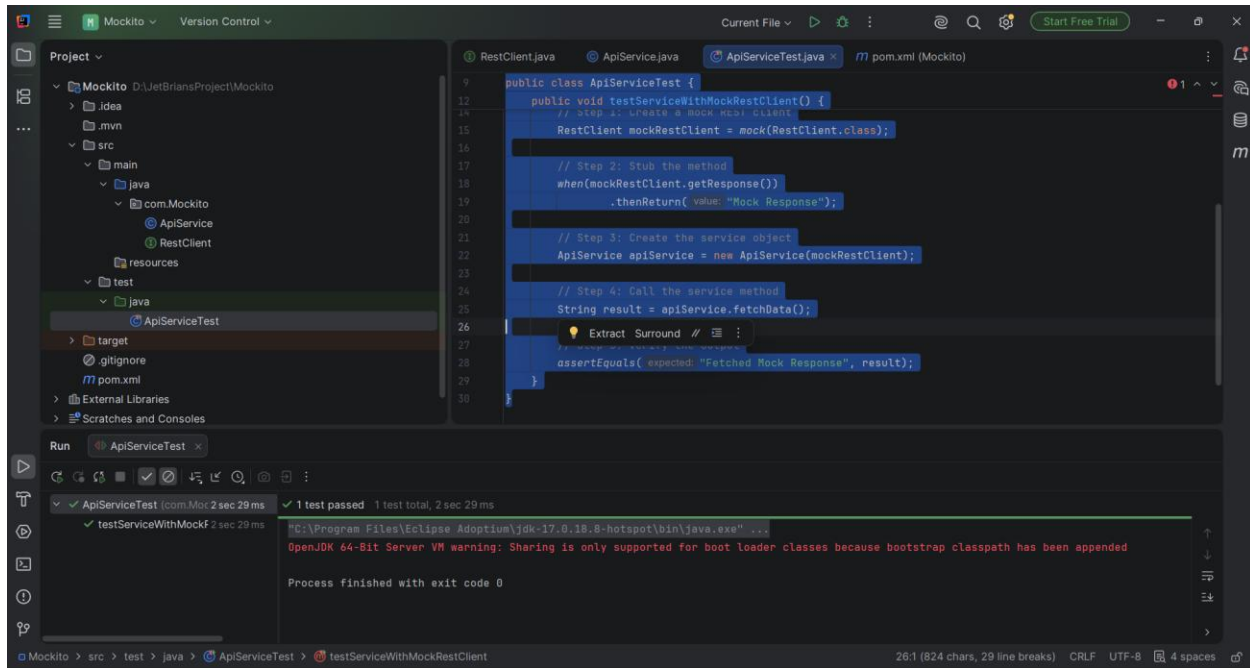
        // Step 3: Create the service object
        ApiService apiService = new ApiService(mockRestClient);

        // Step 4: Call the service method
        String result = apiService.fetchData();

        // Step 5: Verify the output
        assertEquals("Fetched Mock Response", result);
    }

}
```

Output:



The screenshot shows an IDE with a project named 'Mockito'. The project structure includes a 'test' directory with a 'java' subdirectory containing 'ApiServiceTest'. The 'ApiServiceTest' class is open in the editor, showing a test method 'testServiceWithMockRestClient()' that uses Mockito to mock a 'RestClient' and verify the behavior of an 'ApiService'. The test is successful, as indicated by the 'Run' output window showing '1 test passed' and 'Process finished with exit code 0'.

```
public class ApiServiceTest {  
    public void testServiceWithMockRestClient() {  
        // Step 1: Create a mock RestClient  
        RestClient mockRestClient = mock(RestClient.class);  
  
        // Step 2: Stub the method  
        when(mockRestClient.getResponse())  
            .thenReturn("Mock Response");  
  
        // Step 3: Create the service object  
        ApiService apiService = new ApiService(mockRestClient);  
  
        // Step 4: Call the service method  
        String result = apiService.fetchData();  
  
        assertEquals("Expected: 'Mock Response'", result);  
    }  
}
```

Run: ApiServiceTest (com.Mockito) 2 sec 29 ms
✓ 1 test passed 1 test total, 2 sec 29 ms
testServiceWithMockRestClient 2 sec 29 ms
Process finished with exit code 0

Exercise 3: Mocking File I/O

Code:

FileReader.java

```
package com.Mockito;  
  
public interface FileReader {  
    String read();  
}
```

FileWriter.java

```
package com.Mockito;  
  
public interface FileWriter {  
    void write(String data);  
}
```

FileService.java

```
package com.Mockito;

public class FileService {

    private FileReader fileReader;
    private FileWriter fileWriter;

    public FileService(FileReader fileReader,
                       FileWriter fileWriter) {
        this.fileReader = fileReader;
        this.fileWriter = fileWriter;
    }

    public String processFile() {

        String content = fileReader.read();

        fileWriter.write(content);

        return "Processed " + content;
    }
}
```

FileServiceTest.java

```
package com.Mockito;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;

import org.junit.jupiter.api.Test;

public class FileServiceTest {

    @Test
    public void testServiceWithMockFileIO() {

        // Create mock objects
        FileReader mockFileReader = mock(FileReader.class);
        FileWriter mockFileWriter = mock(FileWriter.class);

        // Stub the read() method
        when(mockFileReader.read())
            .thenReturn("Mock File Content");

        // Create the service object
        FileService fileService =
            new FileService(mockFileReader,
                           mockFileWriter);

        // Call the method
        String result = fileService.processFile();
    }
}
```

```

    // Verify the output
    assertEquals("Processed Mock File Content", result);

    // Verify that write() was called
    verify(mockFileWriter)
        .write("Mock File Content");
}
}

```

Output:

The screenshot shows an IDE with the following components:

- Project Explorer:** Shows a project structure with packages `com.Mockito` containing `FileReader`, `FileService`, and `FileWriter`. A test class `FileServiceTest` is located in the `test` package.
- Editor:** Displays the code for `FileServiceTest.java`. The `testServiceWithMockFileIO()` method is highlighted, showing the creation of `FileService`, calling `processFile()`, and verifying the output and `write()` call.
- Run Console:** Shows the execution of `FileServiceTest`. The output indicates that 1 test passed out of 1 test total, taking 907 ms. The process finished with exit code 0.

Exercise 4: Mocking Network Interactions

Code:

NetworkClient.java

```

package com.Mockito;

public interface NetworkClient {

    String connect();

}

```

NetworkService.java

```

package com.Mockito;

public class NetworkService {

```

```

private NetworkClient networkClient;

public NetworkService(NetworkClient networkClient) {
    this.networkClient = networkClient;
}

public String connectToServer() {
    return "Connected to " + networkClient.connect();
}
}

```

NetworkServiceTest.java

```

package com.Mockito;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;

import org.junit.jupiter.api.Test;

public class NetworkServiceTest {

    @Test
    public void testServiceWithMockNetworkClient() {

        // Create a mock NetworkClient
        NetworkClient mockNetworkClient = mock(NetworkClient.class);

        // Stub the connect() method
        when(mockNetworkClient.connect())
            .thenReturn("Mock Connection");

        // Create the service object
        NetworkService networkService =
            new NetworkService(mockNetworkClient);

        // Call the service method
        String result = networkService.connectToServer();

        // Verify the output
        assertEquals("Connected to Mock Connection", result);
    }
}

```

Output:

The screenshot shows an IDE with a project named 'Mockito'. The project structure includes a 'test' directory with a 'java' subdirectory containing 'NetworkServiceTest'. The 'NetworkServiceTest.java' file is open, showing a test method 'testServiceWithMockNetworkClient()' that uses Mockito to mock 'NetworkClient' and verify the output of 'NetworkService.connectToServer()'. The 'Run' button is clicked, and the test passes successfully. The console output shows the test execution details and a warning from the OpenJDK 64-Bit Server VM.

```
public class NetworkServiceTest {  
    public void testServiceWithMockNetworkClient() {  
        // Mock the NetworkClient  
        NetworkClient mockNetworkClient = Mockito.mock(NetworkClient.class);  
  
        // Create the service object  
        NetworkService networkService = new NetworkService(mockNetworkClient);  
  
        // Call the service method  
        String result = networkService.connectToServer();  
  
        // Verify the output  
        assertEquals("expected: 'Connected to Mock Connection', result");  
    }  
}
```

Run: NetworkServiceTest

✓ NetworkServiceTest (com.Mk 947 ms) ✓ 1 test passed 1 test total, 947 ms

testServiceWithMockNetw 947 ms

Process finished with exit code 0

OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

Exercise 5: Mocking Multiple Return Values

Code:

Repository.java

```
package com.Mockito;

public interface Repository {

    String getData();

}
```

Service.java

```
package com.Mockito;

public class Service {

    private Repository repository;

    public Service(Repository repository) {
        this.repository = repository;
    }

    public String processData() {
        return "Processed " + repository.getData();
    }

}
```

MultipleReturnServiceTest.java

```
package com.Mockito;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;

import org.junit.jupiter.api.Test;

public class MultiReturnServiceTest {

    @Test
    public void testServiceWithMultipleReturnValues() {

        // Create a mock repository
        Repository mockRepository = mock(Repository.class);

        // Stub the method with multiple return values
        when(mockRepository.getData())
            .thenReturn("First Mock Data")
            .thenReturn("Second Mock Data");

    }

}
```

```

        // Create the service object
        Service service = new Service(mockRepository);

        // First method call
        String firstResult = service.processData();

        // Second method call
        String secondResult = service.processData();

        // Verify the outputs
        assertEquals("Processed First Mock Data", firstResult);
        assertEquals("Processed Second Mock Data", secondResult);
    }
}

```

Output:

The screenshot shows an IDE with a project named 'Mockito'. The 'Project' view on the left shows the file structure, including 'src/main/java/com/mockito/Service.java' and 'test/java/MultiReturnServiceTest.java'. The 'Run' view at the bottom shows the execution of the 'MultiReturnServiceTest' class, with the output 'Process finished with exit code 0'.

```

public class MultiReturnServiceTest {
    public void testServiceWithMultipleReturnValues() {
        .thenReturn( value: "First Mock Data" );
        .thenReturn( value: "Second Mock Data" );

        // Create the service object
        Service service = new Service(mockRepository);

        // First method call
        String firstResult = service.processData();

        // Second method call
        String secondResult = service.processData();

        assertEquals( expected: "Processed First Mock Data", firstResult );
        assertEquals( expected: "Processed Second Mock Data", secondResult );
    }
}

```

Run: MultiReturnServiceTest

MultiReturnServiceTest (1 sec 53 ms) ✓ 1 test passed 1 test total, 1 sec 53 ms

testServiceWithMulti 1 sec 53 ms

C:\Program Files\Eclipse Adoptium\jdk-17.0.18-hotspot\bin\java.exe ...

OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

Process finished with exit code 0